NYC Citi Bike Demand Forecasting & Optimization

完整项目规划与实施步骤 | Python • Time Series • Mathematical Optimization • Simulation

> 文档定位：纯技术项目实施说明；只列执行步骤与验收标准，不设置时间安排。所有历史骑行数据、模拟库存、优化假设和结果必须严格区分。
>
> 核心问题：在相同的库存、容量、预算和运输假设下，利用各站点下一小时的**已完成骑行数量预测**来指导模拟调运，能否比不调运或简单规则调运获得更好的模拟服务水平与成本表现？

0. 先定义项目边界、研究对象与完成标准

建议初版固定设定：

项目项	初版设定	原因
数据来源	Citi Bike 官方 2025 年 NYC Trip Histories	连续完整年度，覆盖季节性
分析范围	Manhattan 附近约 30 个稳定站点	规模可控；后续可扩张
时间粒度	Station × Hour	可做一小时滚动预测与调运
预测目标	每站下一小时 departures、arrivals	分别预测借车和还车
划分方式	1–8 月训练；9–10 月验证；11–12 月测试	避免随机切分导致时间泄漏
模型	Seasonal Naive、Poisson GLM、XGBoost	保留简单、统计、ML 对照
优化	容量约束的整数线性规划（MILP）	决定站点间调运数量
基线策略	No Rebalancing、Distance Greedy	判断优化是否真正有用
测试	离线滚动仿真、敏感性与消融分析	评价预测及调运效果

最小可交付闭环：真实数据 → 清洗审计 → station-hour 数据集 → 时间序列预测 → 整数规划 → 同条件模拟 → 策略对比 → 可复现实验与报告。

必须承认的限制：官方 Trip Histories 记录已发生/已完成的骑行，**不是**所有潜在借车请求；没有历史库存快照，就不能直接观察 2025 年哪些站点真正空车或满桩。仿真中的缺车、无法还车是特定假设下的模型输出，不能描述为 Citi Bike 真实发生的损失。当前 GBFS 站点容量也不是 2025 年历史容量。

Step 1 — 初始化工程与运行环境

1.1 创建目录

```
citibike_project/
├── data/
│   ├── raw/                # 原始官方 ZIP；不可手工修改
│   ├── interim/            # 临时清洗数据、审计日志
│   └── processed/          # panel、features、stations
├── src/
│   ├── prepare.py          # 数据处理
│   ├── train.py            # 预测模型
│   ├── optimize.py         # 单小时调运模型（完善后）
│   ├── simulate.py         # 策略模拟
│   └── metrics.py          # 统一计算指标（完善后）
├── notebooks/              # EDA 与诊断，不写核心业务逻辑
├── configs/                # 参数与场景设置
├── tests/                  # 自动化测试
├── outputs/
│   ├── figures/
│   ├── tables/
│   └── experiments/
├── reports/                # 最终技术文档
├── requirements.txt
├── README.md
└── .gitignore
```

这是推荐的**最终结构**；随附起步工程暂时只有一部分脚本，不要求第一步就建立所有文件。

1.2 建立虚拟环境（macOS）

```
cd citibike_or_starter
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

建议确认 `python --version`、`pip list`。主要依赖：`pandas`、`numpy`、`scikit-learn`、`xgboost`、`scipy`、`matplotlib`；正式项目可增补 `pyarrow`、`pytest`、`statsmodels`、`geopandas` 等。

1.3 用合成数据验证环境

```
python src/make_demo.py
python src/train.py
python src/simulate.py --hours 48
```

应生成预测指标文件、预测 CSV、仿真汇总表和图表。**这一步只验证程序接口，所有合成数据结果禁止和真实实验混写。**

完成条件：依赖安装成功；示例端到端运行；代码、输入、输出位置明确；Git 忽略原始大文件、虚拟环境与运行缓存。

Step 2 — 下载数据、建立数据清单

2.1 官方来源

1. **Trip Histories：** https://citibikenyc.com/system-data ，进入下载页面选择 2025 年 NYC 月度数据文件，避免混入 Jersey City（常见 `JC` 前缀）数据。

2. **当前站点信息：** https://gbfs.citibikenyc.com/gbfs/en/station_information.json ，可参考站点 ID、坐标、当前 capacity，但不可直接当作 2025 年历史事实。
3. **当前站点实时状态：** https://gbfs.citibikenyc.com/gbfs/en/station_status.json ，后续可以定期采集建立**从采集时刻开始**的库存样本。
4. **可选天气：** https://open-meteo.com/en/docs/historical-weather-api ，必须区分事后观测天气与预测时可用的天气预报。

下载 2025 年 1–12 月 NYC 数据 ZIP，先抽查 1–3 个月文件结构与列名，再补全剩余月份。部分 ZIP 内可能有多个 CSV；不要把 ZIP 与相同内容的解压 CSV 同时输入。

2.2 必须检查的原始字段

字段	含义	使用方式
`ride_id`	行程 ID	去重、质量审计
`started_at`	骑行开始时间	借车事件按小时统计
`ended_at`	骑行结束时间	还车事件按小时统计
`start_station_id`	起始站点	借车站点标识
`end_station_id`	结束站点	还车站点标识
`start_lat` , `start_lng`	起始点经纬度	地理范围与站点坐标核查
`end_lat` , `end_lng`	终点经纬度	站点漂移与空间核查
`rideable_type`	车辆类型	可选分车型实验
`member_casual`	会员 / 临时用户	可选需求分层

2.3 建立下载与覆盖审计

逐月记录：文件名、月份、ZIP/CSV 数量、字节大小、数据行数、时间最小值与最大值、缺失站点 ID 数、重复 ride_id 数、异常行程数、来源 URL、下载日期。必须核查**12 个月都有数据且没有重复输入月份**。

输出： `data/interim/file_manifest.csv` 、 `data/interim/monthly_audit.csv` 。

Step 3 — 清洗、校验与站点筛选

3.1 行程级规则

- 将站点 ID 读取为字符串，防止前导零、数值化或新旧 ID 格式问题。
- 解析起始/结束时间；剔除无法解析、缺失必要站点字段的行程，记录删除数量与原因。
- 计算 `duration = ended_at - started_at` ，剔除结束时间早于开始时间或明显异常的持续时间；阈值属于项目规则，需写入配置并测试敏感性。
- 如果 `ride_id` 有重复，区分同一文件内部重复与跨文件重复，并制定唯一保留规则。
- 验证坐标是否落在合理范围；站点移动、名称变化不能只用名称匹配。
- 对夏令时 DST 做明确的时区说明：本地时间的重复小时与缺失小时需要特殊处理，**不能默认为每天恰好 24 小时**。
- 对缺失的小时不能未经验证就认定“真实零需求”；可能是站点停运、数据未覆盖或采集异常。

3.2 只根据训练期筛选站点

使用 2025 年 1–8 月的站点数据进行候选筛选，避免根据测试期表现挑选有利站点。建议：满足空间范围、活跃月份数与行程量门槛，然后选 30 个最稳定站点。

额外检查：站点重开、迁移和 ID 变更；同一 ID 位置大幅变化要拆分或剔除。简单经纬度矩形框只是 Manhattan 近似，不等于官方行政边界；如要求严格 Manhattan，改用区界多边形空间连接。

输出： `stations.csv`，至少包括 `station_id`、名称、经纬度、训练期样本数、活跃月数、筛选理由、容量来源（`assumed` / `historical_verified`）。

3.3 形成完整 station-hour panel

对于站点 `i`、小时 `t`：

- `departures_it` = `start_station_id == i` 且开始时间落在小时 t 的骑行数。
- `arrivals_it` = `end_station_id == i` 且结束时间落在小时 t 的骑行数。
- `netflow_it = arrivals_it - departures_it`，正值代表该小时已记录流量造成的库存净流入。

将全部选定站点与各个有效小时做笛卡儿积，形成连续 panel；对于可证实站点正常运行但没有骑行的小时填 0，非正常运营 / 无法确认的小时用缺失标记，不要盲目补 0。

样表（虚构）：

station_id	hour	departures	arrivals	active_flag
A	2025-01-06 08:00	11	5	1
A	2025-01-06 09:00	17	9	1
B	2025-01-06 08:00	4	12	1

关键自检：对选中站点汇总的借车数应等于清洗后原始数据中这些站点作为起点的次数；还车数同理。允许区域边界外的骑行跨入/跨出，以免错误丢弃相关库存流量。

输出： `data/processed/panel.parquet`（或起步脚本的 `panel.csv.gz`）、`stations.csv`、`data_quality_summary.csv`。

Step 4 — 探索性分析（EDA）与业务假设

建议至少完成以下图表，并为每张图写清发现和潜在原因：

1. **日级和小时级趋势：**观察全年变化、周期性、节假日与极端波动。
2. **小时 × 星期热力图：**识别通勤早高峰、晚高峰和周末差异。
3. **站点空间分布图：**标出高借出量、高还入量和长期净流入/流出站点。
4. **站点净流量分布：**比较 `arrivals - departures` 的均值、分位数、峰值与持续偏差。
5. **前 30 个站点的需求异质性：**展示热门站点、低需求站点和波动大站点。
6. **异常日期诊断：**查找极端计数是否来自真实天气、异常值、关闭事件或数据问题。

不要只做美观图。EDA 要回答：**为什么需要调运、哪些站点可能互相补充、哪几个时段最值得优化？**

输出： `notebooks/01_eda.ipynb`、`outputs/figures/eda_*.png`、`reports/eda_findings.md`。

Step 5 — 明确预测协议，避免未来信息泄漏

5.1 定义可操作的一小时预测

在小时 t 开始时，预测 `[t,t+1)` 的借出和归还数量。输入只能包括 t 之前已经观测到的信息，以及在 t 时刻确实可取得的日历 / 预测气象信息。

预测目标分别定义为 `y_dep(i,t)` 和 `y_arr(i,t)`；不将净流量作为唯一目标，因为净值可能掩盖高借高还的站点。

5.2 固定数据切分

数据段	时段	使用目的
Train	2025-01-01 至 2025-08-31	模型拟合与特征设计
Validation	2025-09-01 至 2025-10-31	参数选择、模型比较
Test	2025-11-01 至 2025-12-31	一次性最终评估

正式评估允许每小时随着过去的真实骑行记录更新 lag 特征，这叫 **rolling one-step-ahead**；并非在 10 月底一次性预测完整两个月。不能在测试期根据结果调参后仍称测试未触碰。

Step 6 — 特征工程与基线模型

6.1 构造特征

- 日历：`hour_of_day`、`day_of_week`、`month`、`weekend`、节假日标记。
- 周期：`sin(2πhour/24)`、`cos(2πhour/24)`，可加一周周期。
- 历史借车：`dep_lag1`、`dep_lag24`、`dep_lag168`。
- 历史还车：`arr_lag1`、`arr_lag24`、`arr_lag168`。
- 过去窗口：只使用 `shift(1)` 后的 24/168 小时 rolling mean 和 rolling std。
- 站点：Station ID one-hot；可选坐标、训练期站点统计特征。
- 可选天气：实际部署时能在预测时拿到的历史天气 / 预测天气，不能使用目标小时事后的实测天气作为“实时”输入。

任何 scaler、imputer、encoder 的参数只能从训练期拟合；逐小时 rolling 特征必须在数据排序正确后构造并检查边界。

6.2 先建立简单对照

- **Last hour**：用上一小时计数预测。
- **Seasonal Naive**：用同一站点上周同小时（lag 168）预测。
- **Poisson GLM**：研究计数分布与日历效应，可用 `statsmodels`；若过度离散明显，可比较 Negative Binomial。

训练机器学习前保存这些基准结果。若复杂模型不能稳定超过基准，不必人为包装成改进。

输出： `features.parquet`、`forecast_baseline_metrics.csv`、特征字典与泄漏检查结果。

Step 7 — 训练 XGBoost，并完成预测误差评估

- 分别训练 departures 和 arrivals，避免一套输出误认为两个目标相同。
- 可使用 `XGBRegressor`，损失函数选择 `count:poisson` 或其他适合非负计数的形式。

- 参数仅在 Train/Validation 阶段比较：例如 `max_depth`、`learning_rate`、`n_estimators`、`min_child_weight`。
- 选择参数后可在 Train+Validation 上重新拟合，然后固定模型对 Test 产生预测。
- 保存 `pred_dep`、`pred_arr`、实际观测值和基线预测，按 station/hour 对齐；输出必须非负。

计算指标：

- `MAE = mean(|y - ŷ|)`：平均绝对误差，单位“次/站点/小时”。
- `RMSE = sqrt(mean((y - ŷ)²))`：对大误差更敏感。
- `WAPE = Σ|y - ŷ| / Σy`：若分母为零必须单独说明，不能强行除零。

建议分层报告：总计、借车与还车分别、早晚高峰、工作日/周末、高需求站点、低需求站点、11 月和 12 月分开；同时画预测 vs 真实图与残差分布。可以对一小时多站点指标按“小时块”汇总，避免把高度相关的每一行都当独立样本。

运行现有起步脚本：

```
python src/prepare.py --year 2025 --n-stations 30
python src/train.py
```

注意起步版 `train.py` 已包含 Seasonal Naive 和 XGBoost，但 **Poisson GLM、完整分层报告、天气特征仍需自行增加**。

输出： `outputs/forecast_metrics.json`、`outputs/test_predictions.csv.gz`、误差分层表、预测诊断图、训练配置。

Step 8 — 定义数学优化模型与现实假设

预测只是输入。真正的优化决策是：在小时边界，把多少辆单车从站点 i 调往站点 j？

8.1 决策变量和参数

符号	说明
`I_i`	当前**模拟**库存（辆）
`C_i`	站点容量，历史数据不足时**假设**容量
`Dhat_i`, `Ahat_i`	下一小时预测借出、归还量
`x_ij`	从 i 调到 j 的单车数量；非负整数
`d_ij`	两站点距离代理（最初用 Haversine 公里）
`B`	当小时最多可调运的单车总数
`L_i`, `U_i`	站点目标期末库存安全下界和上界
`u_i`, `v_i`	目标区间不足和超出的非负松弛变量

调运后库存：

`Ipost_i = I_i + Σ_j x_ji - Σ_j x_ij`

预测小时末净库存（只是一种**小时聚合近似**）：

`E_i = Ipost_i + Ahat_i - Dhat_i`

定义线性松弛：

`u_i >= L_i - E_i, u_i >= 0`

`v_i >= E_i - U_i, v_i >= 0`

目标：

`min λ_u Σ_i u_i + λ_v Σ_i v_i + λ_d Σ_(i,j) d_ij x_ij`

需要的约束：

- `x_ij >= 0` 且整数，禁止 `x_ii`。
- `0 <= Ipost_i <= C_i`；不能搬走超过库存的单车或把站点调满溢出。
- `Σ_(i,j) x_ij <= B`；如果有卡车容量/数量，还要另加约束。
- 只在允许距离范围内建立调运边；若考虑方向和道路，应换成路线距离。
- 如不允许一小时内对同一辆车反复转运，需要限制中转行为，初版可以直接只允许供给站点→需求站点。

关键理解：这个目标优化的是**预测期末库存平衡与运输距离的代理成本**，不是精确的真实缺车量。小时内是否先借后还会改变实际服务情况，必须由 Step 10 的模拟来评价。

初版示例参数可以是 `capacity=25`、`budget=12 bikes/hour`、`radius=3 km`；罚分权重是场景偏好，**不是实际美元成本**。

8.2 三站点人工算例

例如 `I=[3,27,8]`、`C=[30,30,30]`、`Dhat=[15,5,17]`、`Ahat=[2,3,3]`。先手算各站在不调运情况下的预测期末库存，再比较搬运 1–3 辆车会怎样改变目标函数。检查：容量是否越界、预算是否越界、供给站有没有搬空、目标函数是否下降。**这些数均为假设示例。**

输出： `reports/optimization_formulation.md`、`tests/test_small_network.py`、求解器模型和约束检查。

Step 9 — 实现三种调运策略

对每一个模拟小时，用相同的信息状态和调运限制计算三种策略。

1. **No Rebalancing**：完全不进行调运，用作最低操作成本基线。
2. **Distance Greedy**：先识别预测库存过剩/不足站点，然后按允许距离由近到远调运，直至达到容量或预算约束。
3. **MILP Optimization**：使用 SciPy `scipy.optimize.milp`、OR-Tools 或 HiGHS 求解上一步的数学规划模型。

每个策略需要保存一份 `moves.csv`，字段至少包含 `timestamp`、`strategy`、`source_station`、`target_station`、`bikes_moved`、`distance_km`、`solver_status` 和 `runtime_ms`。

如果 MILP 不可行、超时或失败，**标记求解状态**，不得静默视作最优结果；可以透明地设置 Greedy fallback 并分别统计次数。

检查点：没有负库存、没有超过容量、全部调运整数、总调运量不超预算、无非法距离边、固定输入会得到可复现结果。

Step 10 — 搭建离线仿真并公平比较

10.1 初版：小时聚合仿真

每小时按固定流程：

1. 读取当前**模拟库存**与已经训练好的下一小时预测。
2. 选定一种调运策略，计算 `x_ij`，在小时边界执行**假设的瞬时调运**。
3. 读取测试小时对应的已记录 `departures` / `arrivals`，作为情景中的流量代理。
4. 按明确的事件顺序规则更新库存；无法借出时计为模拟 `unmet_departure`，无法还入时计为模拟 `blocked_arrival`。
5. 更新下一小时库存，并记录调运距离、移动单车数量、库存分布和服务代理指标。
6. 对所有策略使用**同一批测试小时、同一组初始库存和同一套随机数/事件顺序**，确保公平。

起步包的 `simulate.py` 使用小时内随机交错借车与还车，**不具备真实事件顺序**，不能直接视为运营事实；把它当作教学版本的情景模拟即可。

10.2 必须特别处理的因果一致性问题

历史数据中的完整行程有起点和终点：如果一个仿真中的借车请求被拒，那么它原本对应的到达也不应无条件照常发生。把借出量和到达量作为互不相关的计数回放会出现不守恒、重复供车和错误阻止还车等问题。

因此更严谨版本应：

- 用原始 `ride_id` 将开始/结束事件成对关联；
- 未成功发出的模拟借车，不应生成相应归还事件；
- 维护在途单车，处理跨出/跨入研究区域的行程；
- 对受阻还车设定明确的行为规则（例如改投附近站点或记为无法服务）；
- 将真实行程当作“**观测骑行请求的样本路径**”，并说明被拒绝需求不可观测，反事实服务率存在识别限制。

如果暂时不做上述升级，结果标题务必写成 **hourly count-based scenario simulation**，不要叫历史真实缺车率。

10.3 起步脚本调用

```
python src/simulate.py --hours 336 --budget 12 --radius-km 3 --capacity 25
```

`336` 表示 14×24 个模型小时（仅为参数示例）；脚本暂时会使用它自己的固定假设与简化流程。运行脚本前要先有 `test_predictions.csv.gz`。比较结果应在同一套假设下输出，禁止两个策略使用不同需求样本或不同起点库存。

输出： `simulation_hourly.csv`、`simulation_summary.csv`、`moves.csv`（需扩展）、逐小时库存追踪表、策略成本对比图。

Step 11 — 定义评价指标与策略比较方式

11.1 预测指标

MAE、RMSE、WAPE，分 departures 与 arrivals；可额外报告高峰时段指标和按站点的误差分布。

11.2 模拟指标

指标	定义	解释边界
Simulated unmet departures	模拟库存为 0 时无法借出的事件数	**不是现实用户流失量**
Simulated blocked arrivals	模拟库存达到容量时无法还入的事件数	取决于仿真模型
Proxy service rate	1 - simulated_unmet / scenario_departure_requests	仅在定义清楚情景需求时有意义
Bikes moved	每小时调运数量总和	不同策略可比较
Bike-km	Σ 调运数量 × 站点间距离	**不是卡车实际行驶公里数**
Scenario objective	人工惩罚 + 距离代理成本	**不是实际财务成本**
Solver runtime	求解时长与成功率	衡量算法计算可行性

每个策略都提供总体指标和逐小时序列。针对“策略 A 相比 B 的变化”，应报告绝对差与相对变化；当基准为零时不要计算百分比提升。

11.3 统计比较

若用多个天/小时作为比较单位，注意时序相关和同一天内站点相关。可以采用**按天或按周为块的 bootstrap**、重复情景种子与区间估计，而不是把每条站点小时记录当独立样本。重复模拟只量化假设模型内的不确定性，不能消除历史库存缺失导致的结构性偏差。

输出： `policy_comparison.csv`、`comparison_plots.png`、`uncertainty_summary.csv`、指标口径说明。

Step 12 — 稳健性、消融和压力测试

12.1 逐项调整关键假设

变量	可测试值（示例）	想回答的问题
容量假设	15、25、35	结果是否依赖站点容量
调运预算	6、12、24 辆/小时	增加调运资源是否值得
可调运半径	2、3、5 km	距离限制如何影响策略
初始库存	容量的 30%、50%、70%	对初始状态是否敏感
惩罚权重	缺车与满桩多组权重	成本权衡如何变化
随机种子	多组固定 seeds	模拟事件顺序敏感度
测试时段	不同月份、工作日、周末	结论是否普适

12.2 消融实验（Ablation）

- 用 Seasonal Naive 预测替换 XGBoost，其余全部不变；
- 只预测 departures、对 arrivals 使用简单基准；
- 取消或加入天气变量（须满足可获得性条件）；
- 保留相同预测器，仅替换 Greedy/MILP 调运算法；
- 以真实测试期未来流量构造 **Oracle** 理论上界，必须清楚标注不可实际部署。

提前固定情景配置与对照关系。若看到 Test 上最优的参数就继续调参，最终应另留一段真正未使用的数据重新评价；否则把结果诚实称为测试集上的探索性敏感性分析。

输出： `experiments.csv`（每行一个场景及全部参数）、结果汇总透视表、鲁棒性图、失败情景说明。

Step 13 — 自动化测试与数据质量守恒

建议至少覆盖以下测试：

1. 原始 CSV、ZIP 和多文件 ZIP 能正确读取，重复文件不会被双计。
2. 时间解析、异常时长、缺失值和重复 Ride ID 处理可追踪。
3. 选站过程只使用训练期，不泄漏验证/测试站点活跃度。
4. panel 借/还总量与原始清洗行程记录一致（包括跨区域行程）。
5. lag 与 rolling 特征严格来自目标小时之前。
6. 预测 non-negative，站点/时间对齐无重复。
7. 调运不会产生负库存、超过容量、非法边或超预算。
8. 三站点人工算例与 MILP 求解结果一致，求解状态正确记录。

9. 不同策略采用同一模拟样本与随机种子；总量追踪在明确边界下守恒。
10. 固定版本和配置能复现结果；报告统计值从保存文件计算，不手工改写。

输出： `tests/` 测试脚本、`pytest` 测试输出、数据覆盖和实验追踪日志。

Step 14 — 可选升级（在基础闭环完成之后选择）

A. 事件驱动模拟：使用 `ride_id` 和精确开始结束时间回放，维护在途车辆、跨区流量和被拒请求的因果后果；优先级最高，因为它能修正小时随机排序的结构性缺陷。

B. 自建真实库存时间序列：定期保存 GBFS 的 `station_status` 快照。该数据库只覆盖从采集开始的未来时段，无法追溯重建 2025 年库存；同时核查站点 ID、容量变化、采样失败与 API 限流。

C. Rolling-horizon / Stochastic Optimization：预测多个小时的需求场景，在约束下联合优化决策，并比较平均预测与不确定性建模的取舍。

D. Vehicle Routing：加入货车数量、车辆容量、路网行驶时间、装卸时间、司机时限和实际移动路径。此时 `bike-km` 应被更合理的货车成本替代。

E. 网络扩容：将 30 个站扩展到更大地理范围；注意求解器规模、站点营业时间、跨区流入/流出和计算耗时。

这些都是**扩展方向，不是最小项目闭环的前置条件**。

Step 15 — 整理完整交付、复现与技术说明

最终应提供以下内容，全部基于真实运行结果，而不是预先填写数字：

成果	应包含内容
`README.md`	问题、数据来源、环境、命令、结构、假设、复现方法
`DATA_DICTIONARY.md`	输入字段、单位、时间粒度、清洗规则、缺失值规则
`data_quality_summary.csv`	月份覆盖、删行原因、站点筛选与异常
`forecast_metrics.csv`	模型按 train/val/test、借/还、时段分层的指标
`optimization_formulation.md`	决策变量、目标、全部约束、参数含义
`simulation_summary.csv`	各策略服务代理指标、移动量、情景成本
`experiments.csv`	每次实验参数、种子、模型版本、结果路径
`outputs/figures/`	EDA、预测误差、策略对比、稳健性图
`tests/`	单元测试、守恒检查与泄漏检查
`reports/final_report.md`	方法、结果、限制、结论和未来扩展

最终技术报告结构

1. Problem Definition：项目问题、工作范围、需要的假设。
2. Data：数据来源、清洗审计、站点选择与 station-hour panel。
3. Forecasting：目标、特征、切分协议、基线及模型。
4. Optimization：变量、目标函数、约束与求解器。
5. Simulation：运行规则、对照策略、事件与库存处理。

6. Results：真实预测指标、模拟策略差异、成本代理。
7. Robustness：敏感性、消融、区间与失败情景。
8. Limitations：历史缺车不可观测、容量假设、服务需求代理、运输简化。
9. Conclusion：完成的事实、证据支持的结论与后续工作。

完工验收（缺少任一核心项则不能称为端到端完成）

- ☐ 原始数据、筛选条件和每项数据处理规则可以追溯。
- ☐ 预测器相对于简单基线完成时间顺序评估；无已知数据泄漏。
- ☐ 优化变量、约束、权重和求解状态全部记录。
- ☐ 三种策略在相同测试情景中被公平模拟。
- ☐ 主要效果同时展示服务代理、移动成本与不确定性。
- ☐ 所有模型假设和真实数据的可观测边界都有明确说明。
- ☐ 可以从 README 的命令复现至少一套完整运行结果。

官方资源与参考链接

- Citi Bike System Data：https://citibikenyc.com/system-data
- Citi Bike GBFS Station Information：https://gbfs.citibikenyc.com/gbfs/en/station_information.json
- Citi Bike GBFS Station Status：https://gbfs.citibikenyc.com/gbfs/en/station_status.json
- Open-Meteo Historical Weather：https://open-meteo.com/en/docs/historical-weather-api
- SciPy Mixed-Integer Linear Programming：https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html
- XGBoost Python：https://xgboost.readthedocs.io/en/stable/python/python_api.html

代码范围说明：随附起步代码已包含基础清洗、选站、Seasonal Naive/XGBoost、简化 MILP 和小时仿真；本手册中的 Poisson GLM、精确事件驱动仿真、完整统计检验、更多约束、严格区界和实验自动化等均是进一步要实现的工作，不应误认为起步程序已经覆盖。